Agam Sidhu
3027948957
agamsidh@usc.edu
EE 569 Homework #2
February 22, 2026

Problem 1: Edge Detection

I. Motivation

Problem 1(a): Sobel Edge Detection

For this problem, we explore how edges in an image correspond to large intensity changes and how these changes can be used to identify meaningful structures. Edges often represent object boundaries, key structural features, and transitions between different textures. Accurately detecting these edges is important because they capture key visual information while discarding less relevant details. In this problem, we examine how image gradients can be used to highlight such structural information and how adjusting gradient-based parameters affects the quality of the detected edges.

Problem 1(b): Canny Edge Detector:

In this next problem, we are exploring the effect of using a far more advanced edge detection technique: the Canny edge detector. Unlike the simple gradient-based edge detector, which relied solely on gradient magnitude, this approach incorporates gradient edge detection,3 double thresholding, and non-max suppression. These supplemental features allow us to use high and low thresholds to set a clear precedent between weak and strong edges, refine the width of a pixel, and improve localization. Furthermore, by using this type of thresholding, we can keep weak edges that only relate to strong edges, thus reducing noise caused by false detections.

Problem 1(c): Structured Edge:

For our next problem, we are investigating how implementing a Structured Edge approach can adjust our image. In this approach, we see how using a trained Random Forest model can improve on the Canny and Sobel methods by analyzing patterns in the structure in various image patches instead of just focusing on the magnitude of the gradient. This allows the method to highlight meaningful boundaries from the background.

Problem 1(d): Performance Evaluation:

In our final problem, we will be comparing the performances of the Sobel, Canny, and Structured Edge using a quantitative performance metric. The main reason why we are using this is that using a human evaluation benchmark can often be difficult to highlight discrepancies presented

across various images. For our methodology, we are taking in various true edge maps for various images. With these true edge maps, we can categorize our pixels into true positive, false positive, true negative, and false negative values. Based on these values, we can calculate our recall and precision to explain how well our model can accurately identify the true edges whilst avoiding any incorrectly classified edges. In addition to this metric, we also use the F- measure because increasing our threshold tends to reduce our recall but improve our precision. F-measure alleviates this by balancing recall and precision into a single evaluable metric. Now, for our problem, we separately use each ground truth, average precision, and recall for each truth, and then compute the overall F-measure. In addition to this metric, we are also seeing how our threshold value can impact our F-measure.

II. Approach

Problem 1(a): Sobel Edge Detection

Our approach for this problem first required us to convert the color image into a grayscale image. This was denoted by the formula:

$$Y = 0.2989R + 0.5870G + 0.1140B$$

Where:

- R, G, B are the rgb channel values at a given pixel location

After converting into a grayscale image, the image gradients in the horizontal and vertical directions were computed using the Sobel operator. 2 3×3 Sobel kernels were applied to approximate the first-order derivatives in the x and y directions. To handle pixels near the image boundary, border replication was used to ensure valid indexing across the entire image.

The resulting horizontal and vertical gradient maps were then normalized independently using min-max normalization to the range 0–255 so that they could be compared directly. The gradient magnitude image was then calculated by combining the x and y gradients using Euclidean magnitude. Additionally, we normalized the magnitude map from 0 to 255. Lastly, the normalized gradient magnitude image was thresholded to create a binary edge map.

Problem 1(b): Canny Edge Detector:

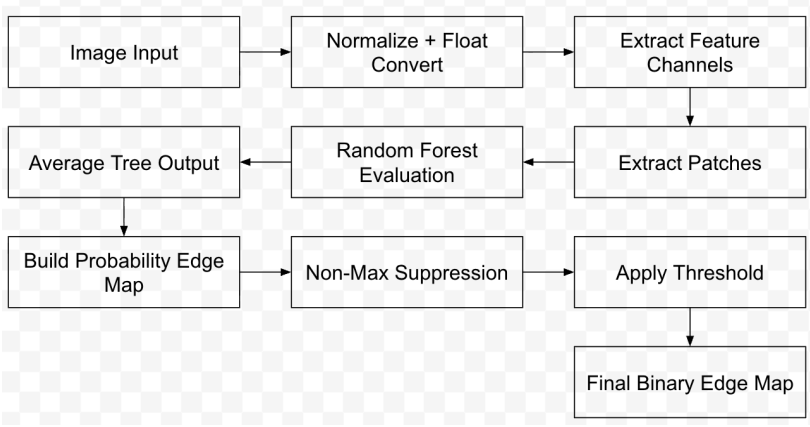
The approach for this problem involved us converting the RGB image into grayscale and then using the same luminance conversion formula as problem 1(a) here to get the intensity at each pixel. Then we clamp it to a range of 0 to 255. Next, we use the Canny edge detector on the grayscale image through the Canny function on OpenCv's library. We also pass low and high threshold values in the command line. The output of the image was a binary edge map where

edge pixels were nonzero. Then we convert it into a single-channel raw file where the image has a white background (255) and edge pixels (with value 0) are black.

Problem 1(c): Structured Edge:

In this problem, we were allowed to use a pretrained model so we used a model from pdollar/edges in the MATLAB toolbox titled modelBsds. We applied this detector model to the Bird and Deer files and in this case, we did not transform the image into a grayscale image. Before processing, we used single precision and normalized the image to 0/1. Also we allowed for non-maximum suppression to create edges that are thin, and well localized. The output was a probability edge map in which each value of a pixel held the likelihood of an edge. To get our final edge map, we added a threshold called percent to a value of 0.20 to preserve boundaries that are “strong”. We also reduced background texture noise through this threshold.

Flow Chart:



As mentioned in the Approach, this approach does not convert the image into grayscale. We first take the image input, normalize it, and apply single precision in the range of 0 to 1. Then we extract feature channels such as the color cues and gradient to describe the structure of the image locally. Then the algorithm extracts patches around each pixel and uses them in the Random Forest operation. In this case, each decision tree predicts an edge response for the given patch. Based on this, the outputs are then averaged to build the probability edge map. After that, we apply Non-Maximum suppression to thin and localize the edges. Lastly, we apply our threshold so that p>= our threshold to create our final binary edge map, which suppresses weak responses while emphasizing strong boundaries.

For this section, we will be talking about the decision tree process. The decision trees are built by splitting the training data on the values of various features, so we separate non-edge samples from edge samples. In each node, a threshold and feature are chosen that best divide the data based on an impurity measure. These can be entropy or Gini indexes. The main object

here is to build child nodes that are more homogenous (similar in terms of the edge classification). Until we hit a stopping condition, such as reaching the max depth of a tree or hitting the minimum number of samples we wish to hit, we continue this splitting procedure. Finally, our leaf nodes at the end store the probability estimation value which highlights the likelihood that a given input corresponds to an edge.

For the Random Forest Classifier, the main principle is to take this procedure of splitting trees in this manner, but combining them into an ensemble. The main approach here is to train multiple decision trees across various random subsets of the training data while also only considering a random set of features. By introducing this randomness, allows us to reduce overfitting while also promoting diversity since the content is randomized. After this, we aggregate the outputs (in this case averaging) to get our final result. In terms of the Structured Edge approach, the Random Forest Classifier helps provide us with a probability estimate for each patch. This, in turn, is combined to give us the final probability edge map, which is better than using a single decision tree, which may struggle with generalization and efficiency.

Problem 1(d): Performance Evaluation:

Now, for our approach to evaluate our Sobel, Canny, and Structured Edges, we will be using the ground truth edges and our Structured Edge toolbox to calculate our objective performance metrics. First, we take our binary edge maps from part 1a (Sobel, part 1b (Canny), and part 1c (Structured Edge). Due to the fact that various threshold values can impact the density of an edge, we evaluate our performance across three different threshold values. Something to note here is that for each threshold, we converted our grayscale edge responses into a binary edge map. Our next step was to compare each ground truth map with the binary edge map. For each pixel, we placed them into a class of True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). True positive relates to whether the detected edge pixel matches a ground truth edge pixel, while a true negative relates to a non-edge pixel being set as a non-edge pixel. A false positive is classified as pixels that are marked as an edge pixel but listed as a non-edge pixel in the ground truth, while a false negative is when we fail to mark an edge pixel from the ground truth class. In order to calculate our Precision and Recall, we use the formulas below:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Furthermore, since we use various ground truth maps for each image, we also aim to calculate the precision and recall separately for each ground truth. Then we take each value and average them across to get our mean recall and precision values. The next step here is to take the averaged values and use them to calculate our F-Measure score so we can have a metric that

highlights both false detections and missed edges. The formula for our F-measure is:

$$F = 2 \cdot \frac{P \cdot R}{P + R}$$

Where: P = precision, R = recall.

After this, we plotted the F-measure values as a function of each detector so we can see how significant threshold selection is in overall performance. For this problem, we used built-in plotting functions with our x- axis = the threshold index and our y-axis = the F- measure that was averaged across all our ground truth maps. Now, for each detector (Sobel, Canny, and Structured Edge), we plot them separately to see how adjusting the threshold can impact the performance.

III. Results

Problem 1(a): Sobel Edge Detection:

Bird Images:

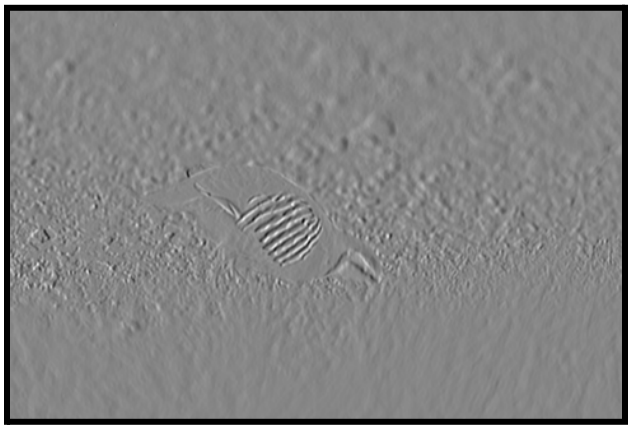


Figure 1: Sobel Gx response for Bird

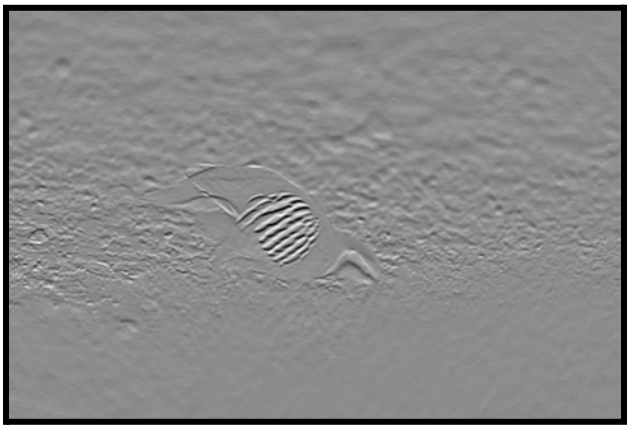


Figure 2: Sobel Gy for Bird

Figure 3: Gradient Magnitude

Figure 4: Binary edge map (Threshold = 25)

Deer Image:

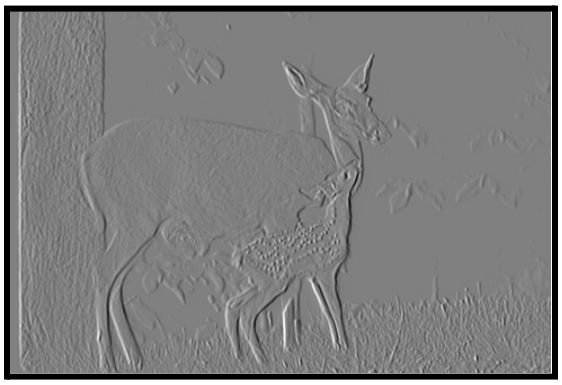


Figure 5: Sobel Gx response for Deer Deer

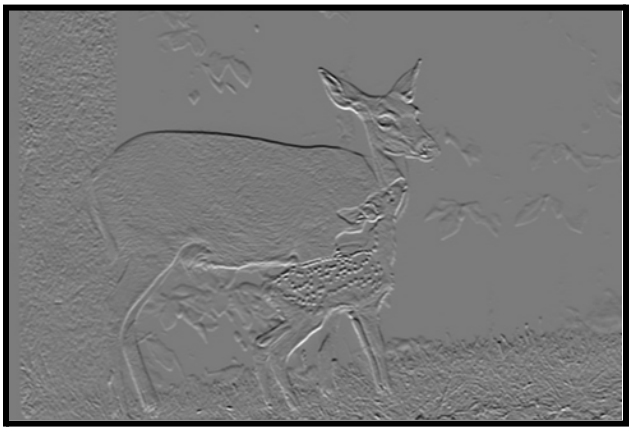


Figure 6: Sobel Gy response for

Figure 6: Gradient Magnitude

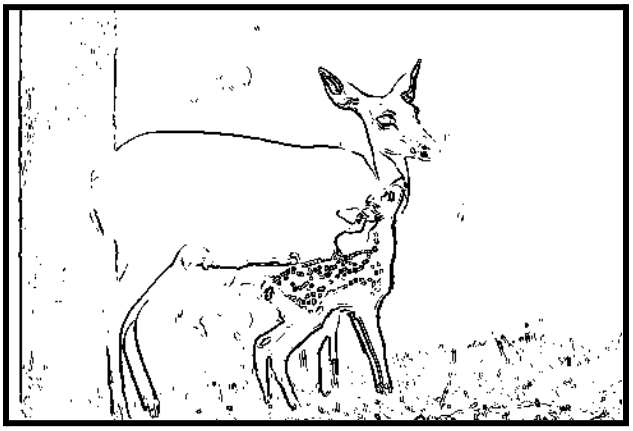


Figure 7: Binary edge map (Threshold = 25)

Problem 1(b): Canny Edge Detector:

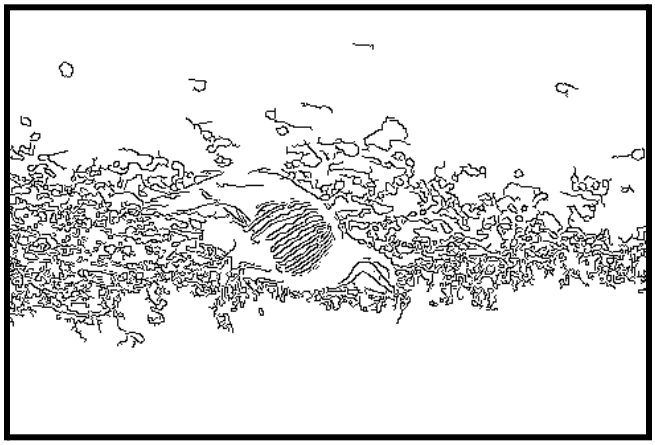


Figure 8: Canny Detector (L=50, H=150)

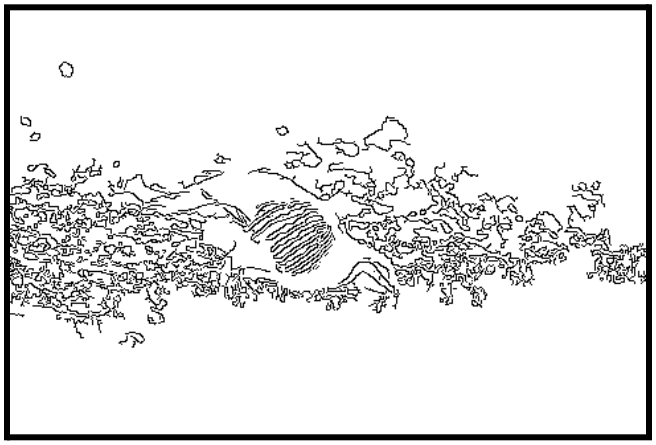


Figure 9: Canny Detector (L=60, H=180)

Figure 10: Canny Detector (L=50, H=150)

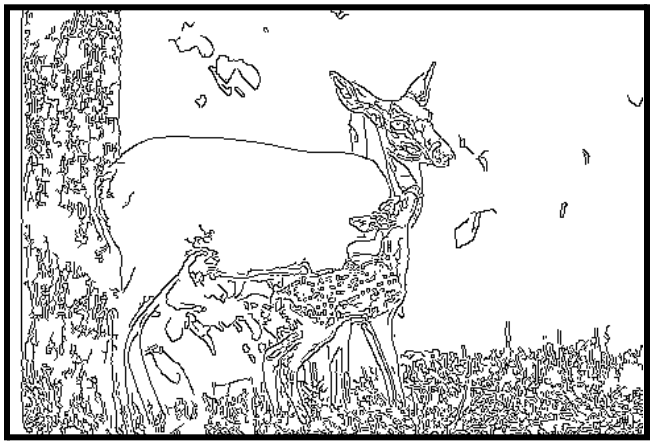


Figure 11: Canny Detector (L=60, H= 180)

Problem 1(c): Structured Edge:

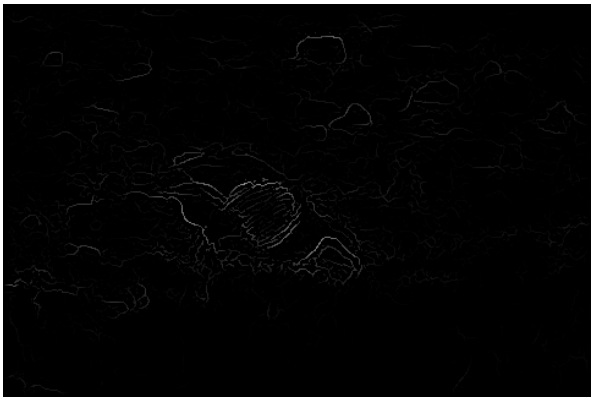


Figure 12: Bird SE Probabilty Map

Figure 13. Bird SE Binary Map (T=20)

Figure 14: Deer SE Probability Map

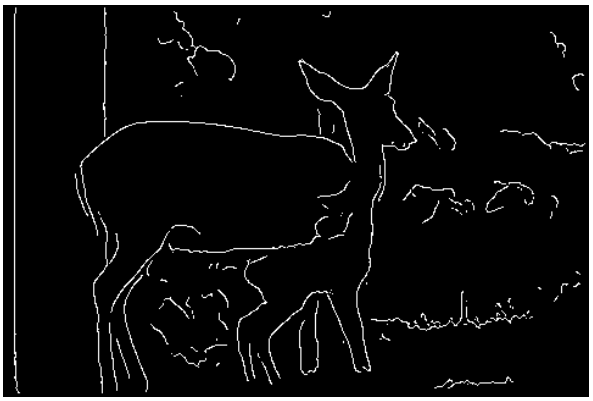


Figure 15: Deer SE Binary Map (T=20)

Problem 1(d): Performance Evaluation:

GT	Sobel_Precision	Sobel_Recall	Canny_Precision	Canny_Recall	SE_Precision	SE_Recall
GT1	0.041416	0.113048	0.043709	0.343457	0.08009	0.029996
GT2	0.03854	0.094601	0.049523	0.343735	0.090925	0.028892
GT3	0.106752	0.168389	0.076712	0.356897	0.286624	0.054351
GT4	0.066605	0.116229	0.064822	0.326256	0.140387	0.034545
GT5	0.050184	0.197809	0.033955	0.40917	0.222799	0.098616

Figure 16: Bird Table

GT	Sobel_Precision	Sobel_Recall	Canny_Precision	Canny_Recall	SE_Precision	SE_Recall
GT1	0.238092	0.392647	0.085111	0.360558	0.387469	0.245531
GT2	0.230823	0.56781	0.064668	0.412762	0.391848	0.370381
GT3	0.313223	0.567158	0.088358	0.415228	0.508804	0.352351
GT4	0.365605	0.448528	0.12703	0.399615	0.564716	0.266817
GT5	0.344495	0.355761	0.141453	0.37653	0.597491	0.243189

Figure 17: Deer Table

Figure 19: Deer Summary Chart

Method	MeanPrecision	MeanRecall	Fmeasure
Sobel	0.298447	0.466381	0.363978
Canny	0.101324	0.392939	0.161105
SE	0.490066	0.295654	0.368808

Method	MeanPrecision	MeanRecall	Fmeasure
Sobel	0.060699	0.138015	0.084316
Canny	0.053744	0.355903	0.093386
SE	0.164165	0.049280	0.075805

Figure 18: Bird Summary Chart

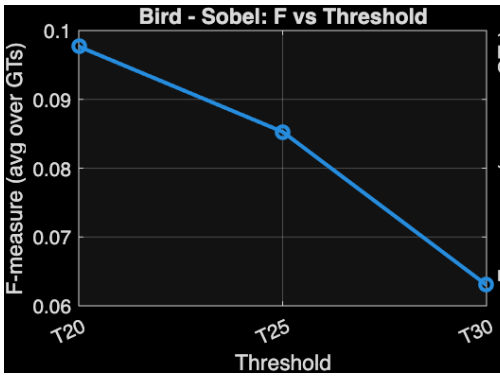


Figure 20: Sobel Graph

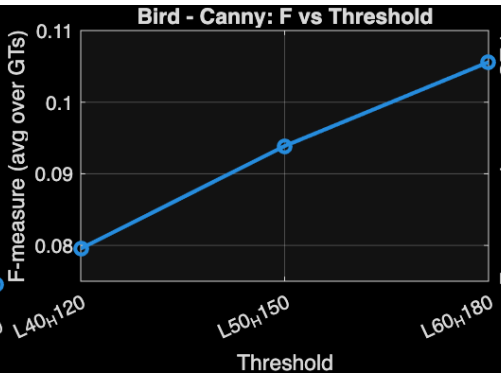


Figure 21: Canny Graph

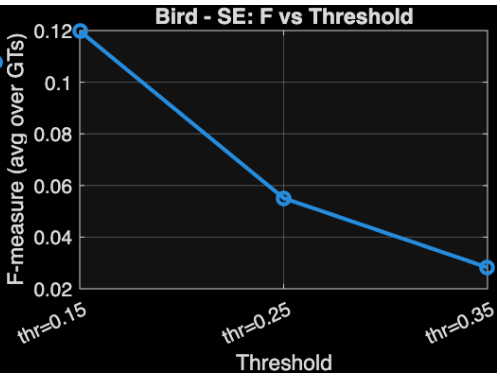


Figure 22: SE Graph

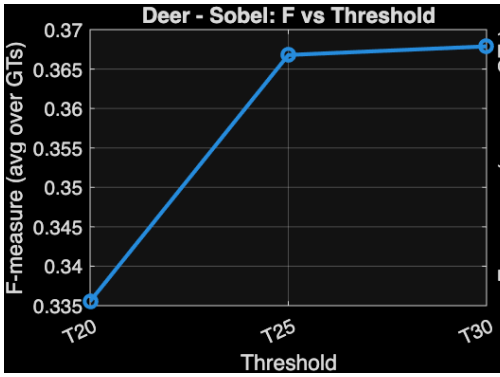


Figure 23: Sobel Graph

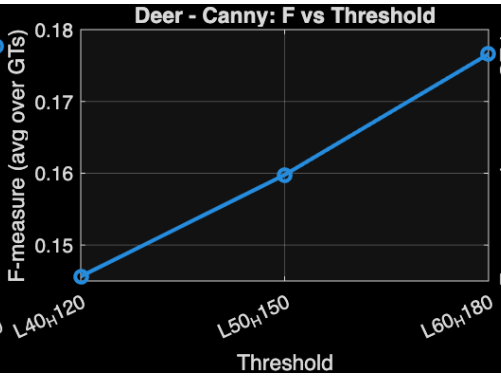


Figure 24: Canny Graph

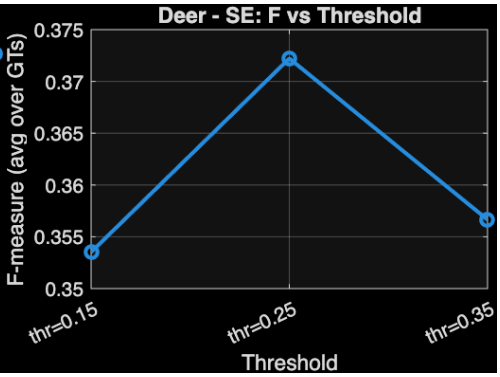


Figure 25: SE Graph

IV. Discussion

Problem 1(a): Sobel Edge Detection:

The results section highlights how adjusting the image gradients can emphasize the boundaries of an image while also suppressing outer regions. For instance, Figures 1 and 2 for the bird and Figures 5 and 6 for the deer show us how adjusting the directional intensity impacts the image. In Figures 1 and 5, where we normalize gx, we can see a stronger outline of the bodies of both the bird and deer while we look up and down the image. In Figures 2 and 6, where we normalize the vertical gradient, we see a stronger outline on the texture and curvature of the animals, emphasizing how the sharp details are visible as we look across the image.

When combining the gradients, we can see an enhanced image where edge information is very clear, to be seen as seen in Figure 6. With this being said, we do have a loud image where smaller details are also highlighted, as seen in Figure 3. This extra noise, however, doesn't make it difficult to see the outline of the bird, making it still a better image. Now, when thresholding the normalized gradient magnitude at 25%, clear edge maps are obtained, preserving the curvature and key details of the animals while reducing excess background noise.

Problem 1(b): Canny Edge Detector:

From the images, we can see how the Canny edge detector approach further refines our images. One reason this can be attributed to is non-maximum suppression. Non-maximum suppression is where pixels are retained if they are the local maximum along the gradient direction. If the pixel is not, we "suppress" it. This process allows us to reduce edges that are thick while also improving our sharpness.

In addition to the non-maximum suppression, the use of low and high threshold values plays a critical role. Pixels that are between the low and high values are considered weaker edges, where they are only retained if they are connected to a strong edge. A strong edge is pixels with gradient magnitudes that are above the high threshold, while pixels with gradient magnitudes below the low threshold are dropped. This can be seen clearly in Figures 10 and 11. Figure 10 contains a lot of fine details as seen on the deer and the tree because it has lower low and high thresholds. Figure 11, on the other hand, reduced these extra details and noise by focusing on the outline of the deer. This can also be seen when comparing Figure 9 to Figure 8, as Figure 9 had less noise and a clearer outline of the bird due to the higher low and high thresholds. All in all, adjusting the high and low thresholds can affect how important minor and fine details are to the overall image. If we want to preserve finer details, introducing a lower low threshold and a lower high threshold would be the proper course of action. However, if we feel that the images have too much noise and seem overly cluttered, then increasing our low and high thresholds is the procedure that should be followed.

Problem 1(c): Structured Edge:

The main parameters mentioned for this portion of the problem include normalizing the image in the range of [0,1], with non-max suppression being enabled so we can improve localization. To choose our T value, we first tried with a lower T value (0.1) and recognized that it resulted in more excess background textures. To further explore the impact, we then changed the threshold to 0.3, which resulted in parts of the main animal background being faded, such as Figure 7. Thus, we applied a threshold value of 0.2 because it still suppressed enough excessive background textures but also preserved any meaningful edge boundaries that were key for the main animal.

After applying the structured edge approach using the modelBsds model, we can see a much more coherent and meaningful image in comparison to the Canny and Sobel approaches. For instance, in Figures 4 and 9 of the bird image, we see lots of background noise creeping into the image, making it difficult to see the full outline of the animal. However, when we look at Figure 13, we are able to see a much clearer outline of the bird with minimal background edges being added into the image. This is mainly because the structured edge approach uses image patches to differentiate the main object from the background. This can be further emphasized when comparing Figure 7 (Sobel) and Figure 11 (Canny) to Figure 15 (Structured Edge). As seen in Figures 7 and 11, we are able to see the main image of the deer and its fawn clearly, but there is copious amounts of background noise making its way into the image. Because of this, the deer looks as if it has an additional pattern on the fur when it actually doesn't. When compared to Figure 14, we can see that even though minor details are still added, the overall boundary consistency is much stronger. Furthermore, the outline of the animals is full and complete while also clear enough to see how the shape of the fawn's head, tilting up, looking at its mother, is distinguishable.

Problem 1(d): Performance Evaluation:

After calculating the precision and recall for each method for the Bird approach (Figure 18), we can see that Canny achieved the highest F-measure with a score of 0.0934. Also, we can highlight that the Canny approach has low precision (0.0537) but high recall, while the Sobel approach also has low precision (0.0607) but a more moderate recall value (0.1380). This means that our Canny method is able to detect a lot of edges (high recall), but it does miscalculate a lot of values and adds a lot of false edges (low precision). The Sobel approach has a similar outcome where there is low precision (so it detects a lot of false edges), but since it does have moderate recall, it detects fewer true edges making it seem worse than the Canny model. Now, when analyzing the Structured Edge approach, we can see that in comparison to the other methods, we have our highest precision (0.1642), but very low recall. In other words, the SE approach only captures/detects edges when it is very confident. So even though it is a good thing where detecting an edge is mostly correct, it misses a lot of real edges, causing our F-measure to drop.

In our other image (the deer), we can see from the table (Figure 19) that our Structured Edge method was able to return the highest F-measure metric in comparison to the Canny and Sobel

methods. The SE approach yielded a score of 0.3668, while Sobel had a close second with a value of 0.3639 and Canny with a measure of 0.1611. These scores begin to make a bit more sense when we incorporate the values from our precision and recall metrics. For instance, the SE approach had a precision of 0.4901 with a recall value of 0.2957. This means that our SE method not only is very likely to detect a true edge when detecting an edge, but it also captures a fair amount of true edges. In our Sobel model, we can see that we have a little bit lower precision (0.2984) with a high recall (0.4664), indicating to us that even though it does detect a lot of true edges, the low precision shows us that there is a good sum of false positives also added. When we compare these methods to the Canny method, we can see that this approach returns low precision (0.1013) and a moderate recall value of 0.3929, highlighting that this model will detect a lot of edges that are mostly all incorrect. Ultimately, when looking to select an approach to complete such a task, I do recommend using the SE approach due to the high F-measure that, unlike the Sobel method, uses high precision with a moderate recall value. The main reason why I do not fully endorse the Sobel method is due to the fact that the Sobel method does introduce a large number of false positives.

After calculating the F-measure for each method, we can see that the best F-measure for the Bird image is the Canny method with a score of 0.0934, which is a very low score. This means that even though the Canny method was our best F-measure while also being at the highest threshold (L60 H180), the image was very difficult to accurately evaluate. Because of this fact, all of the methods, regardless of the threshold value, struggled to achieve high precision and recall. Also, it is important to note that as well increased our threshold for both the Sobel and SE methods, the F-measure decreases. Due to this fact, it is fair to assume that as we reduce recall more than we improve our precision as we increase the threshold.

In our deer images, we can see that both the Sobel and Canny methods, our F -measure increases as we increase the threshold value. The Structured Edge approach, however, worked best at a value of 0.25, indicating that the best balance between precision and recall was found at that threshold. Furthermore, we found that our best F-measure was provided by the done by the threshold value of 0.25 at the SE method (0.3668). As seen from Figures 23–25, we can see that picking the correct threshold plays a significant role in overall performance, and sometimes the most optimal output is one in which the threshold corresponds to a balanced precision and recall.

After analyzing all of the images and data, we can confidently argue that the Deer image is much easier to get a high F-measure. One key argument is that when we compare the highest F-measure between the two images. The deer image's best F-measure was a score of 0.3688, while the Bird image had a score of 0.0934. The main reason for this can be attributed to the fact that the Deer image is a closer build to the ground truth images. If we look at the structural differences in the pictures, we can also see how the deer is easy to outline key object boundaries without saturating the image's background, causing it to be difficult to draw the animal. In the Bird image, we can see that thin structures and real-life noise, such as leaves and feathers, add more noise to the imahe causing difficulty when trying to find true object boundaries. This then leads to low recall and precision create low F- measure scores. On the

flip side, the Deer image is able to highlight the boundaries, making it easier for the edge detection methods to select meaningful edges while also reducing the total number of false positives.

The rationale behind the F-measure definition is that it is able to integrate both precision and recall into a single metric. The main principle here is that a strong model doesn't only pick a large sum of true edges (high recall), but we also want to ensure we are not picking up false edges (high precision). The formula allows us to balance these different but connected values. Now it would not be possible to achieve a high F- measure if our recall is much lower than our precision value or vice versa. For instance, if we had high precision and low recall, then our detector would find a few good edges with very few miscalculated or false edges, but because of the low recall, we would not be detecting enough edges, causing us to miss a large number of edges. Furthermore, if our precision is low but our recall is high, then our detector detects a large number of edges, but since the precision is low, we get a lot of false positives, harming our detector.

If we assume that our sum of recall and precision is a constant, we can use this formula where P + R = C, where p = precision, r = recall, and c = our constant. We can then refer back to our initial F- measure and replace our P + R with the C and R value to be C - P. This leaves us with:

$$F = \frac{2P(C - P)}{C}$$

This means that our F is fully dependent on our P value. The numerator shows us that we will have a downward curve (PC - P^2) where the maximum value is at its center at position c/2. In order to reach our center, we know that then our P = C/2, and since R = C-P, then our R is implied to also equal C/2. So, if our P or R is not equal, then our product (P*R) will be reduced, decreasing our F - measure.

Problem 2:

I. Motivation

Motivation — Problem 2(a): Dithering

Problem 2(a): Dithering:

In this problem, we are exploring how to convert a grayscale image into a half-toned image that can create a perception of intermediate tones when only black and white values are available. The main reason for applying this is that there are machines and printing systems that can't reproduce a full grayscale image directly, so this halftoning approach is used to simulate smooth shading. We explored 3 different techniques: fixed thresholding, random thresholding, and dithering with Bayer matrices. The fixed thresholding approach converts our grayscale image into a binary output that uses a single threshold. Random thresholding uses a random threshold to create a varying distribution of the output values, while the dithering approach leverages Bayer matrices to apply a structured threshold pattern. Using these various techniques, we are exploring how they affect the appearance of the half-toned image.

Problem 2(b): Error Diffusion:

Following problem 2(a), where we explored various half-toned approaches, we are now exploring the impact error diffusion will have on converting a grayscale image. In comparison to the other simple thresholding approaches, error diffusion is able to get a better intensity distribution by distributing the difference between the binary output and the initial grayscale value to neighboring pixels. Here we are exploring 3 different approaches: Floyd-Steinberg, Jarvis-Judice-Ninke (JJN), and Stucki. The key difference in these techniques is that they each have different kernels to distribute the error across the surrounding pixels, which will be highlighted in the approach section.

II. Approach

Problem 2(a): Dithering:

Our approach to our problem first began with us reading the grayscale image (Reflection.raw) and shaping our byte stream into a 2-dimensional matrix of size 1280 x 852 (the dimensions of our image). We also output each of our half-tone results into a separate directory.

Now, for our first approach (Fixed thresholding). We selected our global T value to be 128. This value was selected so we can divide the 256 insight levels into 2 ranges (values less than 128 and greater than or equal to 0, and values greater than or equal to 128 and less than 256), as shown below.

$$G(i,j) = \begin{cases} 0 & \text{if } 0 \leq F(i,j) < T \\ 255 & \text{if } T \leq F(i,j) < 256 \end{cases}$$

For our next approach (random thresholding), we used a random matrix of size 1280 x 852, where each pixel (i,j) would have a random number in the range of 0 to 255. This would be saved in rand(i,j). Now we use a fixed random seed to ensure reproducibility. Once we get our rand(i,j), we compare with the pixel value at that same position.

$$G(i,j) = \begin{cases} 0 & if\ 0 \leq F(i,j) < rand(i,j) \\ 255 & if\ rand(i,j) \leq F(i,j) < 256 \end{cases}$$

This equation above highlights that to see whether a pixel will be mapped to black (0) or white (255), we will compare the grayscale intensity (F(i,j)) to the randomly generated threshold rand(i,j). Now, because we use a uniform distribution for our random threshold, our chance that F(i,j) is greater than rand(i,j) only increases as our pixel intensity increases. This means that darker pixels have a higher chance of being set to 0, while lighter pixels are more likely than not to be set to a value of 255.

For our final approach to this problem, we will use ordered dithering, which uses a structured threshold pattern. Then it takes this threshold pattern and applies it periodically across our image. First, we begin with our Bayer Indexmatrix:

$$I_2(i,j) = \begin{bmatrix} 1 & 2 \\ 3 & 0 \end{bmatrix}$$

Where: I2 is our base 2x2 matrix. Something important here is that the values in this matrix represent the order the pixels are on. This means that 0 is the pixel that is turned on first, 1 = the second pixel, 2 = 3rd pixel, and 3 = the last pixel. In other words, the small index values correlate to the lower threshold, so these areas will be assigned to white earlier as our intensity gets larger.

Now our higher-order Bayer matrices will be generated recursively following this formula:

$$I_{2n}(i,j) = \begin{bmatrix} 4 \times I_n(i,j) + 1 & 4 \times I_n(i,j) + 2 \\ 4 \times I_n(i,j) + 3 & 4 \times I_n(i,j) \end{bmatrix}$$

Where: the In notation is the N x N bayer index matrix that is constructed from smaller matrices. It is key to note that In determines the ranking of pixel positions in the N x N block.

$$T(x,y) = \frac{I_N(x,y) + 0.5}{N^2} \times 255$$

Now, since the Bayer index matrix doesn't represent the intensity threshold, we convert the matrix into a threshold matrix that is scaled between 0 and 255. This transformation can be seen above, where:

- N^2 is the total number of elements
- (x,y) is the position within the matrix
- And 0.5, which centers each threshold within its interval

Now in order to tile the threshold matrix across the image (since the image is much larger than our matrix of size N), we use this modular indexing, which is shown below:

$$G(i,j) = \begin{cases} 0 & if\ F(i,j) \leq T(i\ mod\ N, j\ mod\ N) \\ 255 & otherwise \end{cases}$$

In this equation, we use i mod N and j mod N to get the position within our N x N threshold matrix because the modulus operation lets us repeat in the x and y directions across our image. To simplify it, we choose our threshold value from the N x N matrix and compare it to our grayscale intensity. If our pixel intensity is greater than the threshold at the spatial location, we set it to 255, it will be set to 0. This approach allows us to have more pixel locations turn white as we increase the grayscale intensity based on the order defined by our Bayer matrix. Due to this, the local density of the activated pixels increases with our brightness while preserving our spatial pattern.

Something key to note before we move on is that we are generating Bayer Matrices of sizes N = 2, 8, and 32. At N=2, our matrix is small, and the dither pattern is repeated every 2 pixels in both directions. At N=8, we have 64 positions (8x8) while at N=32, we have 1024 threshold levels. This means that as our N increases, the number of available threshold levels in the block increases.

Problem 2(b): Error Diffusion:

Conversely to problem 2 which used half-toning through threshold-based methods, we are now going to apply error diffusion. The big idea here is that we should be better at preserving tones by propagating the quantization error to the neighboring unprocessed pixels rather than deciding each pixel independently.
Now at each pixel location, we first apply our binary thresholding with T =128:

$$G(i,j) = \begin{cases} 255 & \text{if } F(i,j) \geq 128 \\ 0 & \text{otherwise} \end{cases}$$

Which we then use to compute our error as:

$$e(i,j) = F(i,j) - G(i,j)$$

Now, the key thing here is that we distribute our error to neighboring pixels. Also, we use serpentine scanning, which means that the odd rows are processed left to right while even rows are processed right to left. The diffusion methods follow this approach, so our error is propagated to the future pixels.

$$F(i+\Delta r, j+\Delta c) = F(i+\Delta r, j+\Delta c) + e(i,j) \cdot W(\Delta r, \Delta c)$$

Now our first Error diffusion method is Floyd-Steinberg, which defines the kernel to be :

$$\frac{1}{16}\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 7 \\ 3 & 5 & 1 \end{bmatrix}$$

Where we distribute our error to the right pixel (with the value of 7), and then we send the remaining error to the next row.

Now, our next error diffusion method is the Jarvis-Judice-Ninke kernel

$$\frac{1}{48}\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 7 & 5 \\ 3 & 5 & 7 & 5 & 3 \\ 1 & 3 & 5 & 3 & 1 \end{bmatrix}$$

Where we spread the error two rows below with a wider horizontal distribution.

Our next error diffusion kernel is the Stucki kernel:

$$\frac{1}{42}\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 8 & 4 \\ 2 & 4 & 8 & 4 & 2 \\ 1 & 2 & 4 & 2 & 1 \end{bmatrix}$$

Where we have a similar distribution as the JJN kernel, but we also assign strong weights to nearer neighbors.

III. Results

Problem 2(a): Dithering:

Figure 1: Original Reflection Image

Figure 2: Fixed Threshold Dithering

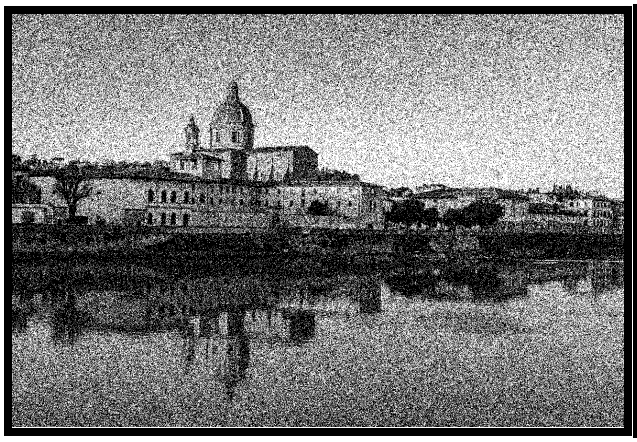


Figure 3: Random Threshold Dithering

Figure 4: Ordered Dithering (N=2)

Figure 5: Ordered Dithering (N=8)

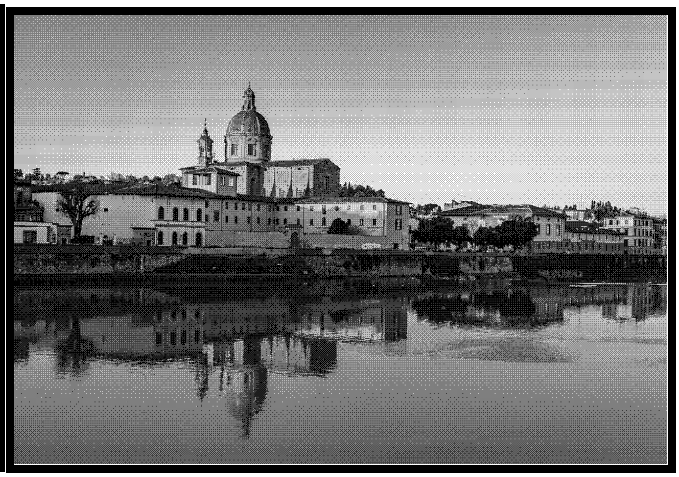


Figure 6: Ordered Dithering (N=32)

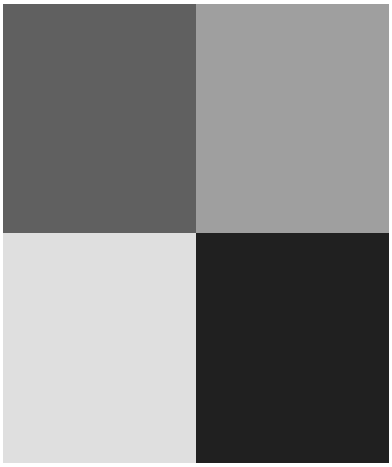


Figure 7: Bayer Threshold(N=2)

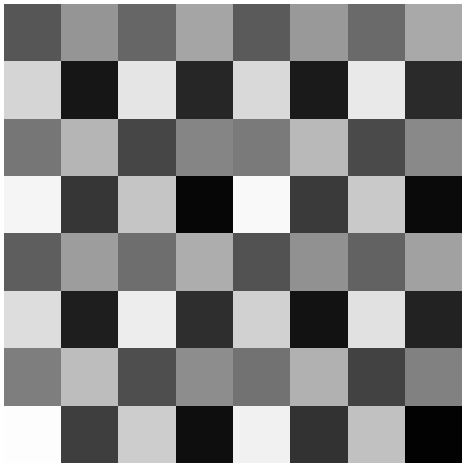


Figure 8: Bayer Threshold(N=8)

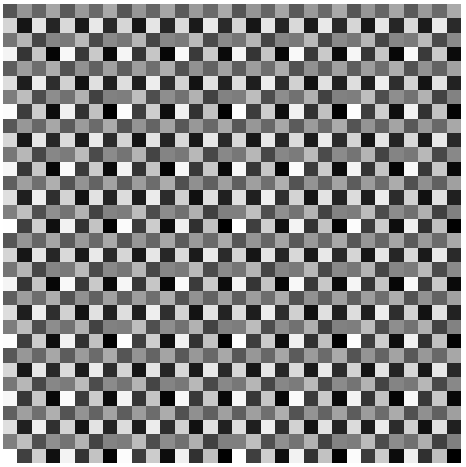


Figure 9: Bayer Threshold(N=32)

Problem 2(b): Error Diffusion:

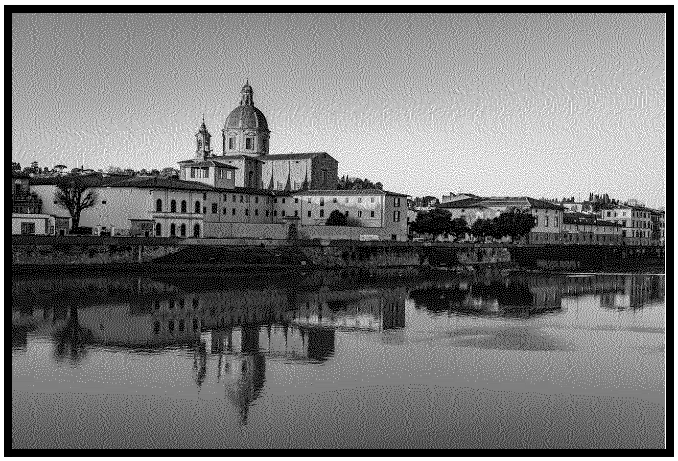


Figure 10: Floyd Steinberg Error Diffusion

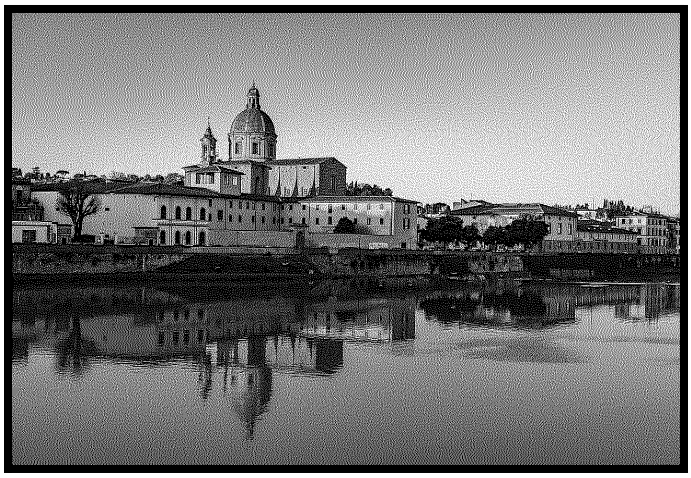


Figure 11: JJN Error Diffusion

Figure 12: Stucki Erro Diffusion

IV. Discussion

Problem 2(a): Dithering:

So after generating the fixed, random, and ordered thresholding, we are able to see how each differs. From the fixed thresholding approach, we can see how the image has strong white and black tones, showing that this transformation struggles to output mid-tones (Figure 2). In our random thresholding output, we can see that even though it is able to capture tones better than the fixed thresholding, the overall approach has a lot more “noise” with the collection of many disruptive pixels on the image (Figure 3). In comparison to those two, our order dithering method provided us with less noise and a better grasp of the midtones represented in the image. Now as we increase our matrix size from 2 to 32, we can also see how the resolution improves and that the reflection in the water in Figure 6 is clearer than in Figures 5 and 4. An

example of this can be seen when comparing the reflection of the upper tower's windows in the water. This small detail, which is emphasized in Figure 6, in comparison to Figures 4 and 5, is why we can ultimately conclude that as we increased our N to 32, our final resolution became more defined. In conclusion, our dithering approach was our most successful in capturing small details, mid-tones, and reducing the overall noise that may arise when applying this transformation.

Problem 2(b): Error Diffusion:

In this problem, we can see that they all have smoother tone transitions in comparison to the dithering matrix in part A. This is mainly because the quant error is spread to neighboring pixels, thus preserving our local intensity. When comparing different error diffusion models, we can see slight differences. First, we can see that the Floyd Steinberg method, despite the overall detail preservation, does see more grainy noise in Figure 10. When we look at JJN, it does a better job in silence. More of the grain is noisy, as shown in Figure 11, but minor details are harder to spot. For instance, in comparison to Figure 10, Figure 11 has details such as the extra pole on the top of the tower distorted in the image, highlighting some of the potential trade-offs for applying such a strategy to an image. When we applied the Stucki error diffusion matrix, we can see the extra pole(in Figure 12) in better definition in comparison to Figures 10 and 11, while also reducing the noise that can be seen in the other approaches we followed.

Now, when we compare our various error diffusion methods to our dithering matrix methods in part A, we can notice several varying differences. First of all, our error diffusion methods tend to carry a little bit more noise (with a grainy texture) across the image, as seen in Figures 10, 11, and 12, while the dithering matrix method reduces this noise, as shown in Figures 4, 5, and 6. Now, despite this excess noise, I would argue that the error diffusion matrix methods are more irregular in certain details, which replicate the original image more than the dithering matrix. A clear example of this is when comparing the tones on the window in Figures 10, 11, and 12. The windows that are being hit by the sunlight are each different tones, as seen in Figure 1. Conversely, in Figures 4, 5, and 6, all of the windows replicate the same tone, emphasizing the trade-off in selecting this method. Ultimately, if I had to choose one method, I would choose the Stucki approach as it still preserves finite details and irregularities that are found in pictures. Even though the dithering matrix approach is computationally cheaper and easier to implement, preserving the tone structure is more important in my eyes. Losing finite details for a quicker run program is not a valuable trade-off.

A potential improvement to this method is to adapt the diffusion strength based on the local characteristics. For instance, in areas with low gradient intensities, we can spread the error a bit more so we can reduce the grain noise that is commonly output in this approach. In areas that have high gradient regions, we can limit the error so we do not blur any sharp transitions, such as the window tone or the pole on the top of the building. All in all, this approach, in theory,y would reduce my biggest issue with the error diffusion approach (the noise) while preserving the fine details (something this approach is very good at already).

Problem 3:

I. Motivation

Problem 3(a): Separable Error Diffusion:

Our main goal for this problem is to explore how the process of using error diffusion changes from grayscale images to color images. The one method we are applying here is to take the R G B channels found in color images and apply the Floyd-Steinberg algorithm on each channel. Because of this algorithm, our output for a given channel will be one of 8 colors (white, yellow, cyan, magenta, green, red, blue, black). Overall, the problem is to see how applying error diffusion to each channel independently can affect the color representation.

Problem 3(b): MBVQ Diffusion:

In this problem, we will be exploring another error diffusion method: Minimum Brightness Variation Quadrants (MBVQ). This approach entails us splitting the RGB color space and then picking vertices that lower our overall brightness variation. The reasoning behind this is that we can preserve the consistency of our colors while also reducing any noise added by the separate error diffusion approach.

II. Approach

Problem 3(a): Separable Error Diffusion:

For this problem, we first take our color scale image and break it down into 3 individual channels (RGB). Now each channel here is then considered as a grayscale image in which we apply the same Floyd-Steinberg error diffusion approach we did in problem 2(b). Now, if we break this further down, we apply binary thresholding with a value of 128. Based on the difference between our binary output and the original value, we save it as our quantization error. We use this error and distribute it to neighboring pixels that have yet to be processed with the Floyd-Steinberg matrix. In our final step, after processing all 3 channels, we recombine the channels to get our final output image. It is also key to note here that since our channel can only be 0 or 255, the output can be one of 8 possible colors.

Problem 3(b): MBVQ Diffusion:

For this problem, we first take our original RGB image and, based on our Minimum Brightness Variation Quadrants, we partition the image into one of the 6 regions. The formula for the corrected RGB value can be seen below:

$$\mathrm{RGB}\,(x, y) + e\,(x, y)$$

Where: e(x, y) is the error calculated from processed pixels. This is then used to find the closest vertex (V) in the MBVQ tetrahedron. This vertex refers to one of the possible RGB vertices mentioned in part a. We add this as the output value at the pixel location. Once we select our new vertex V, we calculate our new quantization error with the formula below:

$$RGB\ (x, y) + e\ (x, y) - V$$

Where: The RGB is the original value, e is the previous quantization error from neighboring pixels, and vertex V is the closest vertex at location (x,y). Once we do this, we take the error and distribute it to unprocessed neighboring pixels with Floyd-Steinberg error diffusion. We continue this process until all pixels have been processed, returning our final reconstructed image.

III. Results

Problem 3(a): Separable Error Diffusion:

Figure 1: Original Flowers

Figure 2: Floyd-Steinberg Error Diffusion

Problem 3(b): MBVQ Diffusion:

Figure 3: MBVQ Floyd-Stienberg Error Diffusion

IV. Discussion

Problem 3(a): Separable Error Diffusion:

After applying our separate error diffusion approach, there are some key things to note in the output image. The resultant image does a good job preserving the overall structure of the image (the flowers and their petals) in Figure 1, while still preserving most of the major colors present in Figure 1. Despite preserving these aspects, the output seems to have grainy noise sprinkled around the image, distorting the background of the pedals and some of the finer details of the flowers as seen in Figure 2. The main shortcoming of this approach is that we ignore any aspect of correlation between RGB channels during the error diffusion process. Because we process each channel independently, any minor issues generated can combine and make more unnatural colors. This issue can then bleed into creating inconsistencies in brightness and create noise in otherwise smooth portions of an image.

Problem 3(b): MBVQ Diffusion:

As mentioned above, a major shortcoming of the separable error diffusion approach is that we process the RGB channels independently, which causes us to introduce noise and distort fine details of our image. Fortunately, the MBVQ approach can address this by considering the overall color of the pixel and selecting a vertex that minimizes the brightness variation. In other words, by constraining the color outputs into a specific region of the RGB cube, we are able to maintain the initial relationship between each channel. Thus, reducing the excess noise and errors produced, especially in areas that have a smoother transition. When comparing the two Figures (2 and 3), it is easy to see how Figure 3 is the better method. One example of this can be seen with the grassy background. Because of the separable error diffusion approach, the grass background in Figure 2 has erratic patches with more color speckling. Conversely to Figure 2, Figure 3 has a closer color distribution to Figure 1 (the original image). Also, Figure 3 has a smoother color transition from black into green, as stressed by the emphasis on the tones of the colors, while Figure 2 has louder color shifts. In conclusion, the MBVQ approach has a better output image that correlates to the initial image in comparison to the separable error diffusion method.